KCC CODE CLUB · GAME MAKERS

Week 5 Lesson Plan

Variables and Score

Big idea: A variable is a labelled box. You can put a number in it, look at what's inside, and change it while the program runs.

At a glance

New concepts	Variables, `set` vs `change`, scope (lightly)
New blocks	`set [var] to`, `change [var] by`, `show variable`, `hide variable`, the variable reporter
Class activity	Countdown Timer
Project checkpoint	The game has a score. Good catches add, bad catches subtract
Homework	High score; make the score look good

Before the class

- Print the Week 5 handout.
- A cardboard box with "SCORE" written on it, and a stack of number cards. Genuinely worth the two minutes it takes to make — the physical prop does more for this concept than any explanation.
- Skeleton `Sky Catcher — end of Week 4` ready.

0:00–0:05 — Show & Tell

Ask for the bad-object homework specifically, and take two. Several of them will have made something funny.

0:05–0:15 — Live demo: making a variable

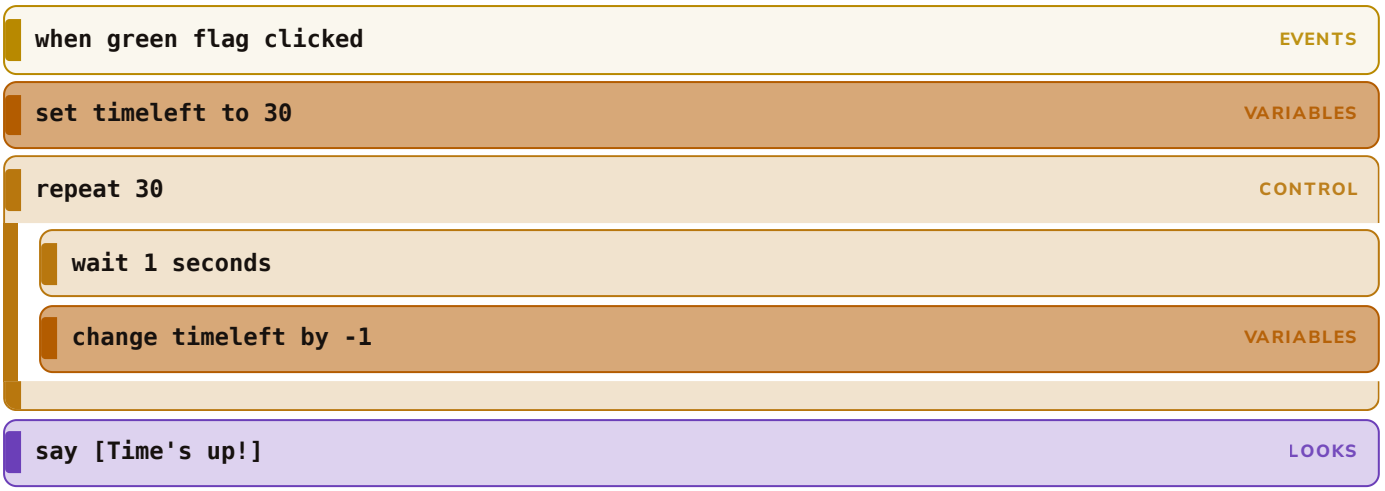
Frame it first, in one sentence: *"a variable is a box with a label on it.* `set` *throws away what's inside and puts something new in.* `change` *adds to what's already there."* Make them tell you the difference back before you build anything — this distinction is the whole class, and it's where today's bugs come from.

1. Variables category → **Make a Variable** → name it `score`. Naming matters; make them use lowercase, no spaces, and a name that says what it holds. `score`, not `s` or `thing`.
2. The variable now shows on the stage in a little box. Drag it somewhere sensible.
3. `when green flag clicked` → `set score to 0`. Explain that **every game starts by resetting its variables**, and that forgetting this is why their score will start at 47 next time.
4. `when [space] key pressed` → `change score by 1`. Press space repeatedly. Watch it climb.
5. Drop the `score` reporter into a `say` block: `say (score)`. Show that a variable can be used anywhere a number can.
6. **Deliberate mistake:** put `change score by 1` inside a `forever` loop with no condition. Run it. The score explodes into the thousands. Look horrified, then explain — *this exact bug is coming for you today.*

Number 6 is this week's inoculation. Do not skip it.

0:15–0:30 — Class activity: Countdown Timer

The demo already built a counter that goes up. This one goes down, inside a loop, which is the part they haven't seen.



Fast finishers go to bonus challenge 3, Click Counter, and bolt it onto the timer: how many clicks in thirty seconds? That's a complete little game built from two variables, and several of them will realise it's something they could put in their real project.

0:30–0:55 — Project build: score

Goal: **"By the end of today your game keeps score."**

1. Make a variable called `score`.
2. Under the green flag on the **player** sprite (or the stage), add `set score to 0`.
3. In the good object's catch `if`, add `change score by 1`.
4. In the bad object's catch `if`, add `change score by -1` and a different sound.
5. Position the score display top-left. Right-click it for large readout if they want.

The bug they will all hit: the score jumps by ten or twenty per catch. This is because the `if <touching player?>` stays true for several trips through the loop while the object is still overlapping. The fix is the `go to` immediately after — moving the object away makes the condition false again. If they've put the `change score by 1` *after* the `go to`, or forgotten the `go to` entirely, the score runs away.

Let them hit it. Then ask: *"How many times is your computer checking whether you're touching? And how long are you touching for?"* It's a great bug because the explanation is satisfying.

Then playtest. Swap seats and play each other's. Compare high scores. The competitive energy this releases is the best thing that happens all term.

Save.

0:55–1:00 — Wrap

Save, back up, homework.

Homework

1. **High score.** Add a second variable, `highscore`, that only updates when `score` beats it. (Hint: `if <(score) > (highscore)> then [set highscore to (score)]`. Note that high score must **not** be reset on the green flag, or it isn't a high score.)
2. **Make it look good.** Position your score display sensibly, try the large readout, and hide any variables the player doesn't need to see.

Bonus challenges

1. Make the game get faster as your score rises — use `score` inside the `wait` in the falling loop. (Careful: waits can't be negative.)
2. Add a `lives` variable that starts at 3 and goes down when you catch a bad object. What should happen at zero? See if you can make something happen. (This is next week's lesson — a genuine head start.)
3. **Click Counter.** `when green flag clicked` → `set clicks to 0`; `when this sprite clicked` → `change clicks by 1`; then `if <(clicks) = 10> then say [You win!]`. Then combine it with the countdown timer: how many clicks can you get in thirty seconds?
4. Add a `level` variable that goes up every ten points, and show it on the stage.

Common bugs this week

Symptom	Cause
Score jumps by 10+ per catch	The condition stays true for several loops. The object must move away immediately
Score starts at whatever it was last time	Missing `set score to 0` under the green flag
Score goes up but never down	The bad object's `if` is checking the wrong sprite, or missing entirely
Variable box isn't on the stage	It's unticked in the Variables palette, or `hide variable` was used
Two variables with almost the same name	They made a duplicate by accident. Delete the spare — right-click in the palette
High score resets every game	They put `set highscore to 0` under the green flag

Notes for the second volunteer

`set` versus `change` is the whole class. When a child is stuck, don't look at their code first — ask them "do you want to replace what's in the box, or add to it?" Nine times out of ten they'll answer correctly and then fix their own block.

Links

- Grow a dragonfly and Drum star — Raspberry Pi Foundation, both variable-heavy
- Skeleton: `Sky Catcher — end of Week 5` in the club studio